

Fine-tuning MACE with the mdstats campaign CLI

The campaign CLI turns source-certified VASP data into a current selected training set and a fresh production model. It keeps scientific authorities, restart state, replay identity, checkpoint evidence, and currentness checks in one disk-backed campaign.

Run commands from the repository root:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Current workflow

The public scientific lifecycle is exactly:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

Post-production qualification of the finished product is a separate downstream family:

```
qualification status | qualification run | qualification activate-locked
```

storage is an orthogonal artifact-management command. status reports the training lifecycle, and advance chooses the next safe owner from that lifecycle; neither introduces another scientific state machine, and advance never runs qualification or opens locked evidence.

1. Create a configuration

```
python tools/mdstats-mlff-campaign.py --config campaign.toml init
```

Set the paths and foundation identity in the generated file. A minimal campaign provides a training root, an inspected foundation checkpoint, and one selected replay corpus:

```
[paths]
training_root = "/path/to/LTA_training"
foundation_model = "/path/to/mace-foundation.model"
replay_set = "/path/to/replay.extxyz"
```

The generator exposes the current target-size policy explicitly:

```
[target_data.size_convergence]
target_size_power_min = 7
target_size_power_max = 14
evaluation_size_powers = [8, 9, 10]
fidelity_epochs = [1, 3, 10]
```

The power ceiling is configuration, not a hidden fixed-size scientific limit. The available population still bounds materializable candidates. Optimizer seeds are authored only by the sole enabled training method.

Current post-selection CV settings are authored under one canonical section:

```
[post_selection.cv]
fold_count = 2
partition_seed = 7
seeds = [11]
max_num_epochs = 2
acceptance_maximum = 0.5

[post_selection.production]
seeds = [5]
```

Pre-target fold controls are not generated as target-size authority. Historical read-only fields may be accepted for a compatible existing campaign, but they cannot silently become current scientific settings.

The learned model supports single (FP32) or double (FP64) precision. mdstats scientific reductions, reference fitting, geometry, and persistent bookkeeping remain FP64 in either mode. The selected acceleration backend and MACE runtime identity are bound by doctor and the campaign protocol.

2. Check inputs and runtime

```
python tools/mdstats-mlff-campaign.py --config campaign.toml doctor
```

Do not continue until blocking source, manifest, foundation, replay, backend, and runtime checks pass. doctor records the manifest approval and the exact runtime realization used by later owners. A missing accelerator or unavailable long-production environment is reported as unavailable; it is not converted into a qualification pass.

3. Prepare the current substrate

Review the source manifest first:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

When the manifest needs operator approval, approve that exact digest:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare --approve-manifest
```

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

Use --refresh-inferences before approval when source metadata or inference inputs changed. Preparation is restartable and owns only the current neutral substrate:

```
source/frame/label authority
-> protected statistical relations
-> one P_train/M3 split and pi_train/pi_eval
-> one common target-size preparation
```

prepare does not select a target size, train a candidate, rank a checkpoint, or publish production. The configured ladder is an experiment definition, not a decision. The one-time destructive cutover rejects obsolete derived target-size records before reuse and quarantines them rather than migrating them. Those records are never translated into current authority.

Preparation reuses only lower-level inputs that current owners revalidate. A source or scientific-identity change invalidates the affected generation; provenance-only presentation changes do not change arithmetic. Interrupted work is resumed by rerunning the same command after inspecting status.

4. Select the target size

```
python tools/mdstats-mlff-campaign.py --config campaign.toml select-target-size
```

This is the only current screening entrypoint. It is the sole command that trains target-size candidates or decides `N_selected`. Every candidate is an exact prefix of one deterministic `pi_train` order, so the selected set is

```
T_selected = pi_train[:N_selected]
```

The screen uses the configured power range, direct nested evaluation populations `M1 subset M2 subset M3`, the configured `fidelity_epochs`, and the ordered seeds from the sole enabled training method. It runs the authenticated continuation:

```
n1 / M1 -> n2 / M2 -> n3 / M3
```

The current default is $(n1, n2, n3) = (1, 3, 10)$. Boundaries are continuation points; an earlier better checkpoint cannot replace the prescribed endpoint. The reducer first narrows the qualified population, then freezes one size and its exact membership or records a typed scientific failure.

Replay metrics, post-selection CV, physical-observable evidence, and downstream qualification cannot rank or tie-break a size. A terminal nonconvergence at the configured ceiling is a scientific result, not permission to invent a rescue size. An incomplete but nonterminal run remains resumable.

The selected size and membership are not editable fields. Every current read re-derives them from authenticated reducer state and `pi_train`; divergence fails closed.

5. Validate the frozen method

```
python tools/mdstats-mlff-campaign.py --config campaign.toml cross-validate
```

Cross-validation starts only after selection and consumes exactly `T_selected`. It validates the training method, not the amount of data. The configured `K >= 2` folds preserve the P1 split-exclusion and correlation relations; every required fold and optimizer seed must pass the target-only acceptance predicate.

Fold partitions are constructed inside the already frozen selected set. A fold may fit training-only transforms from its own training partition, freezes its representative on its authorized monitor, and evaluates the held-out partition only afterwards. Held-out fold results cannot change `N_selected`, membership, checkpoint policy, or the method definition. Replay remains a separate admissibility/retention concern and supplies no ranking credit.

A missing or failing fold is a methodological failure. It leaves selection evidence unchanged and does not authorize final production; it is not replaced by a mean, majority, best-seed, or partial-fold result.

6. Train fresh final production

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train-production
```

Final production starts from the accepted foundation with fresh optimizer, RNG, and run state. It trains the complete exact `T_selected` using the method accepted by cross-validation and `[training].max_num_epochs`. Screening and CV checkpoints are not production parents, even when their numeric seed or size matches.

The production horizon is independent of the screen's `n3`. A production-only configuration change invalidates production descendants while leaving the selected binding and accepted CV evidence current. Both post-selection owners re-authenticate currentness before work and publish only under a commit-time currentness fence.

The training lifecycle ends at fresh final-production closure. Everything after it validates that finished product without being able to change it.

7. Qualify the frozen product

Qualification consumes the final-production publication that train-production already froze. It never creates, reorders, or shrinks that publication, and it owns no target-size, cross-validation, production, checkpoint, seed, or member decision. Every threshold under `[qualification]` is fixed in the configuration before any product outcome is observed.

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification status
```

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification run
```

`qualification run` executes or resumes the nonlocked components for the exact frozen publication:

- **deployment parity** - the published model is exported, converted to the deployed ML-IAP artifact, executed through the real supported LAMMPS runtime, and compared against the authenticated in-framework model under dtype-justified tolerances;
- **local PES** - deterministic symmetric displacement modes on a candidate-independent `OUTER_MONITOR` base cohort, checked for pointwise force agreement, restoring sign, and stiffness/curvature against matched external references;

- **relaxation** - fixed-cell minimization compared against matched reference relaxations, with protected-topology safety judged separately from geometric fidelity;
- **dynamics** - bounded NVT warm-up and NVE propagation through the deployed artifact, checked for temperature behaviour, energy drift, minimum pair distance, force bounds, and persistent topology damage;
- **calibration** - uncertainty calibration of the exact frozen committee on the reserved UNCERTAINTY_CALIBRATION role, or an explicit not_applicable for a single-model product with no accepted uncertainty estimator.

Three outcomes are not failures and must not be read as one:

- `waiting_for_reference` means the external first-principles evidence the frozen physical plan asked for has not been supplied. The exact request is written to the reference root as `reference-request.json`; supply a matching `reference-bundle.json` and rerun. Nothing is ever passed on absent evidence.
- `not_applicable` means the frozen policy declares a component inapplicable to this product.
- an *unavailable* supported deployment runtime blocks the deployment claim rather than passing or rejecting it.

A component rejection rejects that exact published product. It never selects a different seed, checkpoint, or committee member, never shrinks a committee, and never reaches back into target-size selection, cross-validation acceptance, or production training.

The one-shot locked test

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification activate-locked --confirm
```

This is the only path that opens the reserved LOCKED_INTERPOLATION_TEST cohort. It requires `--confirm`, requires every mandatory nonlocked component to have already passed, and is refused a second time for the same publication and cohort. Activation is irreversible: once the cohort is revealed it is never a fresh locked test again, whatever the policy is later changed to, and a retrained product needs genuinely new independent evidence.

A locked pass produces the terminal `release_qualified` verdict; a locked failure rejects the exact published product.

Where the evidence lives

<code>campaign/.mdstats/qualification/g<N>/objects/</code>	immutable release evidence
<code>campaign/.mdstats/qualification/g<N>/attempts/</code>	attempt state and scratch
<code>campaign/qualification-references/<plan>/</code>	reference request and bundle

Durable qualification evidence is release evidence, not reconstructible scratch: storage cleanup never reclaims it, and it also cannot reclaim an artifact an in-flight qualification attempt still references.

Inspect and resume

```
python tools/mdstats-mlff-campaign.py --config campaign.toml status
python tools/mdstats-mlff-campaign.py --config campaign.toml advance
```

The workspace stores the durable campaign state and content-addressed payloads:

```
campaign/
|-- campaign-manifest.json
|-- .mdstats/campaign.sqlite3
|-- .mdstats/                # current-generation records and caches
|-- runs/                    # authorized checkpoints and logs
|-- models/                  # current production model evidence
`-- results/                 # bounded summaries and cleanup reports
```

Rerunning a current owner is safe: complete authenticated cells are reused, stale or corrupt derived caches are rebuilt, and a superseded owner cannot publish a current descendant. Close stores/processes cleanly before inspecting or rerunning another owner.

Storage management

Storage is orthogonal to scientific lifecycle. Start with a read-only report:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage report
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier cache --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe
```

Use a dry run before cleanup. Transitional P6/P7 storage provides storage report, storage cleanup --tier safe, and storage cleanup --tier cache. In P6/P7, both safe and cache cleanup perform zero acceleration-cache eviction, retaining performance and model caches like frame-cache and checkpoint-model-cache. Consequential operations (recompute, compaction, archival, deduplication) are deferred to the post-P7 storage reset (CODE-MLFF-CAMPAIGN-STORAGE-IO-RESET1). Cleanup protects external inputs, current scientific records, selected checkpoints, restart evidence, and diagnostics.

Interpreting outcomes and limitations

The durable result is the authenticated chain of source identity, neutral substrate, target-size experiment, selected binding, CV acceptance, and final production identity. A target-size scientific failure is terminal evidence and does not expose a production next action. A missing accelerator, unavailable target-machine run, or absent downstream qualification is reported as deferred or unavailable rather than silently passed.

The P6 implementation provides current functional, restart, and public-surface closure. It does not establish GPU, long real-data, M-ladder decision- preservation, or downstream release qualification.